Primeira Lista de Exercícios

Tópicos Avançados de Redes Neurais Profundas e Aplicações
Prof. Gilson Antonio Giraldi

Lorran de Araújo Durães Soares

23 de julho de 2026

Introdução

Este documento apresenta a resolução da primeira lista de exercícios da disciplina *Tópicos Avançados em Redes Neurais Profundas e Aplicações*, ministrada pelo professor Gilson Antonio Giraldi. A lista é composta por três questões, que serão apresentadas ao longo deste documento, seguidas de suas respectivas resoluções.

O código desenvolvido para os experimentos deste trabalho está disponível no link `https://drive.google.com/drive/folders/1oJJ2vFglwy4Y1LzfdJWO_xj0TzPgdin2?usp=sharing`. A implementação foi realizada em arquivos *Jupyter Notebook* (formato `.ipynb`), utilizando a biblioteca NumPy [2] para manipulação de arrays numéricos e realização de operações de pré-processamento dos dados, a biblioteca Matplotlib [3] para visualização dos resultados, incluindo gráficos da evolução do treinamento, e a biblioteca PyTorch [6] para a implementação, treinamento e avaliação dos modelos de aprendizado profundo.

1 GANs para Parabolóide

Questão 1: Considere o parabolóide $P \subset \mathbb{R}^3$ representado por:

$$S : \mathbb{R}^2 \longrightarrow P; \quad S(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2 + 1) \tag{1}$$

Construa o conjunto de dados:

$$D = \{S(x_i^1, x_i^2) \in \mathbb{R}^3; \quad i = 1, \ldots, N\}$$

(a) Usando APENAS o conjunto de dados D, gere um modelo GAN que compute S;

(b) Tomando o gerador G, projete uma estratégia para avaliar a qualidade da representação através do erro quadrático médio (RMSE) e do erro absoluto médio (MAE). Discuta os resultados;

(c) Calcule ∇G através de diferenciação automática sobre um conjunto de pontos Z no espaço latente. Tomando a expressão (1), calcule ∇S analiticamente sobre Z e compare os resultados. Discuta seus achados.

Resolução:

Primeiramente, para a resolução desta questão, foi gerado o conjunto de dados D a partir do parabolóide definido na Equação 1. Utilizando a biblioteca NumPy [2], foram amostrados 5000 pontos do domínio $\mathcal{X} = [-2, 2] \times [-2, 2]$ a partir de uma distribuição uniforme. Cada ponto $(x_1, x_2) \in \mathcal{X}$ foi mapeado pela

função $S(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2 + 1)$, gerando o conjunto de dados $D = \{S(x_i^1, x_i^2)\}_{i=1}^{5000} \subset \mathbb{R}^3$. A Figura 1 ilustra os pontos amostrados no plano cartesiano e a superfície do parabolóide correspondente.

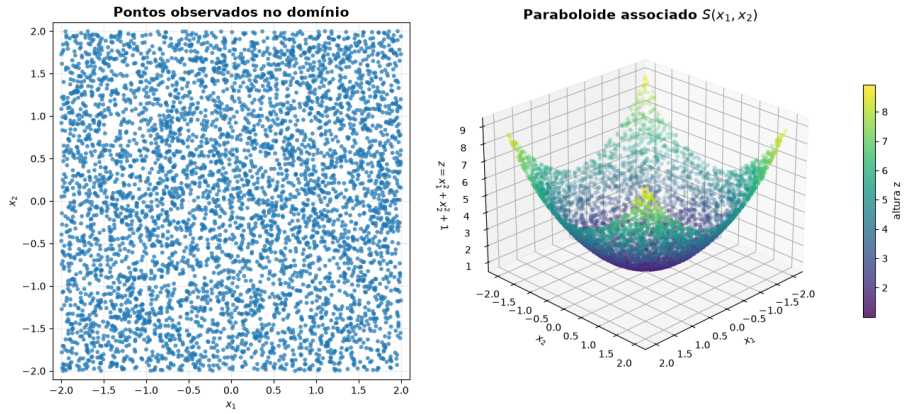


Figura 1: Pontos do domínio $\mathcal{X}$ amostrados uniformemente e o respectivo parabolóide gerado.

Desse conjunto, 80% das amostras foram destinadas ao treinamento da GAN, enquanto os 20% restantes foram utilizados como conjunto de teste. Como o conjunto de dados possui três coordenadas, sendo a terceira definida pela soma dos quadrados das duas primeiras acrescida de uma constante, seus valores tendem naturalmente a apresentar uma escala maior. Para minimizar esse desbalanceamento entre as variáveis de entrada, os dados foram normalizados utilizando a padronização Z-Score:

$$z = \frac{x - \mu}{\sigma}, \tag{2}$$

em que μ e σ representam, respectivamente, a média e o desvio padrão de cada coordenada, calculados apenas sobre o conjunto de treinamento.

Este exercício utiliza uma rede do tipo GAN (*Generative Adversarial Network*), originalmente proposta por [1]. Após a preparação dos dados, foi definida a arquitetura empregada neste trabalho. A Figura 2 apresenta as arquiteturas das redes Geradora e Discriminadora, inspiradas em arquiteturas já consolidadas na literatura.

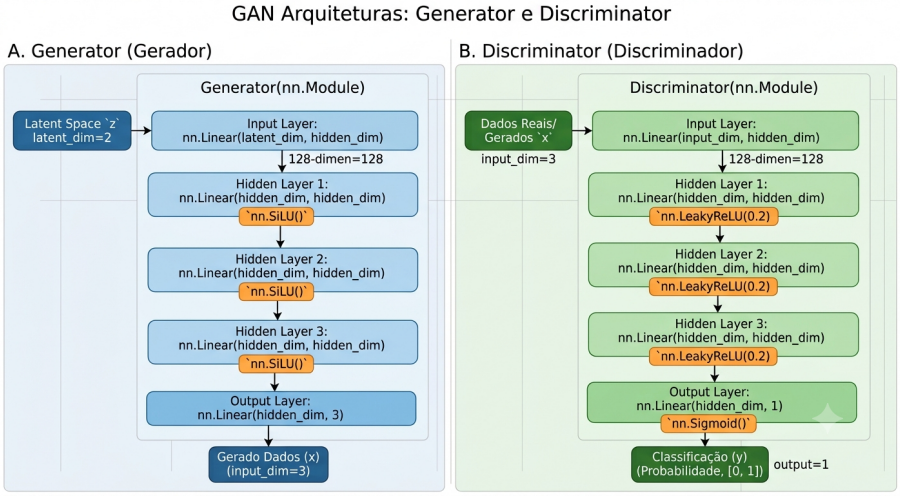


Figura 2: Arquiteturas da rede Geradora e da rede Discriminadora.

A rede Geradora é composta por cinco camadas totalmente conectadas: quatro camadas ocultas, cada uma com 128 neurônios, seguidas da função de ativação SiLU, definida na Equação 3, e uma camada de saída com três neurônios, correspondentes às coordenadas (x_1, x_2, x_3) do parabolóide gerado. A utilização da função SiLU foi definida empiricamente, com base nos resultados obtidos durante os experimentos.

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}, \tag{3}$$

A rede Discriminadora também é composta por quatro camadas ocultas com 128 neurônios cada. Nessas camadas é utilizada a função de ativação LeakyReLU, definida na Equação 4:

$$\text{LeakyReLU}(x) = \max(\alpha x, x), \tag{4}$$

em que $\alpha = 0.2$. Assim como na rede Geradora, a escolha dessa função de ativação e o seu hiperparâmetro foi realizada empiricamente, considerando o desempenho observado durante os experimentos. A camada de saída é composta por um único neurônio, seguido da função de ativação sigmoide, definida na Equação 5:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \tag{5}$$

A saída da rede Discriminadora, denotada por $D(x) \in [0, 1]$, representa a probabilidade estimada de uma amostra pertencer à distribuição dos dados reais. Dessa forma, a utilização da função de ativação sigmoide permite interpretar a saída como uma probabilidade: valores próximos de 1 indicam maior confiança de que a amostra seja proveniente do conjunto de dados reais, enquanto valores próximos de 0 indicam maior confiança de que a amostra tenha sido gerada pela rede Geradora.

Para a otimização dos parâmetros das redes Geradora e Discriminadora foi utilizado o algoritmo Adam (*Adaptive Moment Estimation*). Foi empregada uma taxa de aprendizado de 10^{-4}, enquanto os hiperparâmetros $\beta_1 = 0.9$ e $\beta_2 = 0.999$ foram definidos empiricamente com base no desempenho observado durante os experimentos. O algoritmo Adam realiza a atualização dos parâmetros de acordo com as Equações 6–9:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \tag{6}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \tag{7}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \tag{8}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \tag{9}$$

em que g_t representa o gradiente da função de custo em relação aos parâmetros θ no instante t, m_t e v_t correspondem, respectivamente, às estimativas do primeiro e do segundo momentos dos gradientes, $\hat{m}_t$ e $\hat{v}_t$ são as versões corrigidas dessas estimativas, η é a taxa de aprendizado e ε é uma constante positiva utilizada para garantir a estabilidade numérica durante a atualização dos parâmetros.

A função de custo utilizada no treinamento da GAN foi baseada na *Binary Cross-Entropy* (BCE), uma função amplamente empregada em problemas de classificação binária, definida na Equação 10. Sua utilização é adequada nesse contexto, pois a rede Discriminadora realiza uma tarefa de classificação

binária, distinguindo amostras reais das amostras geradas pela rede Geradora. Enquanto o discriminador busca classificar corretamente essas amostras, o gerador é treinado para produzir dados sintéticos capazes de serem classificados pelo discriminador como pertencentes ao conjunto de dados reais.

$$\mathcal{L}_{BCE}(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]. \tag{10}$$

Durante o treinamento da GAN, o discriminador é otimizado para classificar amostras reais como pertencentes à classe $y = 1$ e amostras geradas pelo gerador como pertencentes à classe $y = 0$. Assim, sua função de custo é definida por:

$$\mathcal{L}_D = -\frac{1}{B} \sum_{i=1}^{B} \left[\log(D(x^{(i)})) + \log\left(1 - D(G(z^{(i)}))\right)\right], \tag{11}$$

em que $x^{(i)}$ representa uma amostra real do conjunto de treinamento, $z^{(i)}$ é um vetor amostrado do espaço latente, $G(\cdot)$ representa a rede Geradora, $D(\cdot)$ representa a rede Discriminadora e B é o tamanho do lote de treinamento.

Por outro lado, a rede Geradora é treinada para produzir amostras que maximizem a probabilidade de serem classificadas pelo discriminador como reais. Para isso, foi utilizada a função de custo não saturante (*non-saturating loss*), definida por:

$$\mathcal{L}_G = -\frac{1}{B} \sum_{i=1}^{B} \log\left(D(G(z^{(i)}))\right). \tag{12}$$

Essa formulação fornece gradientes mais informativos durante o treinamento, especialmente nas etapas iniciais, contribuindo para uma convergência mais estável da rede Geradora.

O procedimento completo de treinamento adotado é apresentado no Algoritmo 1. O treinamento foi realizado durante 300 épocas sem critérios de parada, utilizando $k = 1$, em que k representa o número de atualizações da rede Discriminadora realizadas para cada atualização da rede Geradora em cada lote (*batch*) de treinamento. Durante o treinamento, a rede Geradora recebe como entrada vetores latentes $z \in \mathbb{R}^d$, com $d = 2$, amostrados da distribuição uniforme $p_z = U(-2, 2)$. Esses vetores pertencem a um espaço latente independente do espaço dos dados e são utilizados como fonte de aleatoriedade para a geração de novas amostras. A partir desses vetores, a rede Geradora aprende uma transformação capaz de produzir amostras que seguem a distribuição dos dados reais.

Após o treinamento, a Figura 3 apresenta a evolução das funções de custo das redes Geradora e Discriminadora ao longo das épocas. Observa-se que, nas primeiras épocas, a perda da rede Geradora aumenta, enquanto a perda da rede Discriminadora diminui, indicando que, inicialmente, a rede Discriminadora consegue distinguir com facilidade as amostras reais das amostras geradas.

À medida que o treinamento evolui, a rede Geradora passa a produzir amostras cada vez mais semelhantes às amostras reais, dificultando a tarefa da rede Discriminadora. Como consequência, a perda da rede Geradora tende a diminuir, enquanto a perda da rede Discriminadora aumenta e posteriormente se estabiliza em torno de um determinado valor.

Além disso, observa-se que ambas as funções de custo apresentam oscilações ao longo de todo o treinamento. Esse comportamento é esperado em redes adversariais, uma vez que as duas redes são otimizadas simultaneamente em um jogo de soma zero: melhorias em uma delas tendem a alterar o desempenho

Algorithm 1 Treinamento da GAN

Require: Conjunto de treinamento D, gerador G, discriminador D, número de épocas N, tamanho do lote B, número de atualizações do discriminador $k = 1$

for época $= 1$ até N **do**
  for cada lote $\{x^{(i)}\}_{i=1}^{B}$ **do**
    for $j = 1$ até k **do**
      Amostrar um lote de exemplos reais $\{x^{(i)}\}_{i=1}^{B}$
      Amostrar um lote de vetores latentes $\{z^{(i)}\}_{i=1}^{B} \sim p_z$
      Gerar amostras falsas $\hat{x}^{(i)} = G(z^{(i)})$
      Calcular

$$\mathcal{L}_D = -\frac{1}{B}\sum_{i=1}^{B}\left[\log D(x^{(i)}) + \log\left(1 - D(\hat{x}^{(i)})\right)\right]$$

      Atualizar os parâmetros da rede Discriminadora utilizando Adam
    end for
    Amostrar um novo lote de vetores latentes $\{z^{(i)}\}_{i=1}^{B} \sim p_z$
    Gerar amostras falsas $\hat{x}^{(i)} = G(z^{(i)})$
    Calcular

$$\mathcal{L}_G = -\frac{1}{B}\sum_{i=1}^{B}\log D(\hat{x}^{(i)})$$

    Atualizar os parâmetros da rede Geradora utilizando Adam
  end for
end for

da outra. Assim, as oscilações refletem o equilíbrio dinâmico entre a capacidade da rede Geradora de produzir amostras mais realistas e a capacidade da rede Discriminadora de distingui-las das amostras reais.

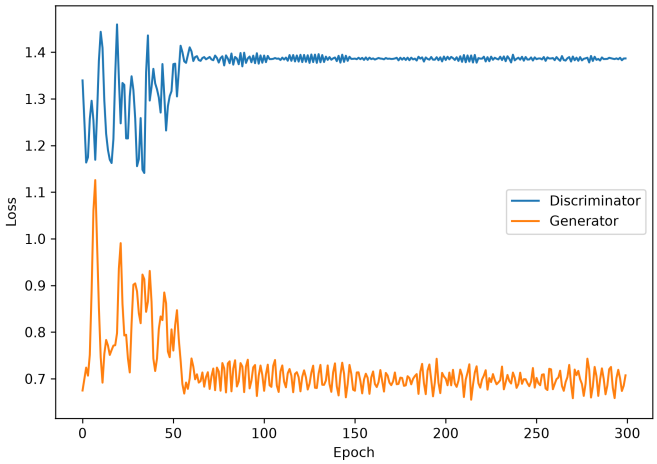


Figura 3: Evolução das funções de perda da rede geradora e da rede discriminadora.

Após o treinamento, a rede Geradora foi utilizada para produzir novas amostras a partir de um conjunto de vetores aleatórios bidimensionais, cujas coordenadas foram amostradas independentemente de uma distribuição uniforme $p_z = U(-2, 2)$. O número de vetores amostrados foi definido como igual ao número de pontos presentes no conjunto de teste.

A superfície correspondente ao conjunto de dados original é apresentada à esquerda da Figura 4, enquanto as amostras geradas pela rede Geradora são apresentadas à direita da mesma figura. Visualmente, observa-se que a GAN foi capaz de aprender a geometria do parabolóide, produzindo uma distribuição de pontos bastante semelhante àquela observada no conjunto de dados original.

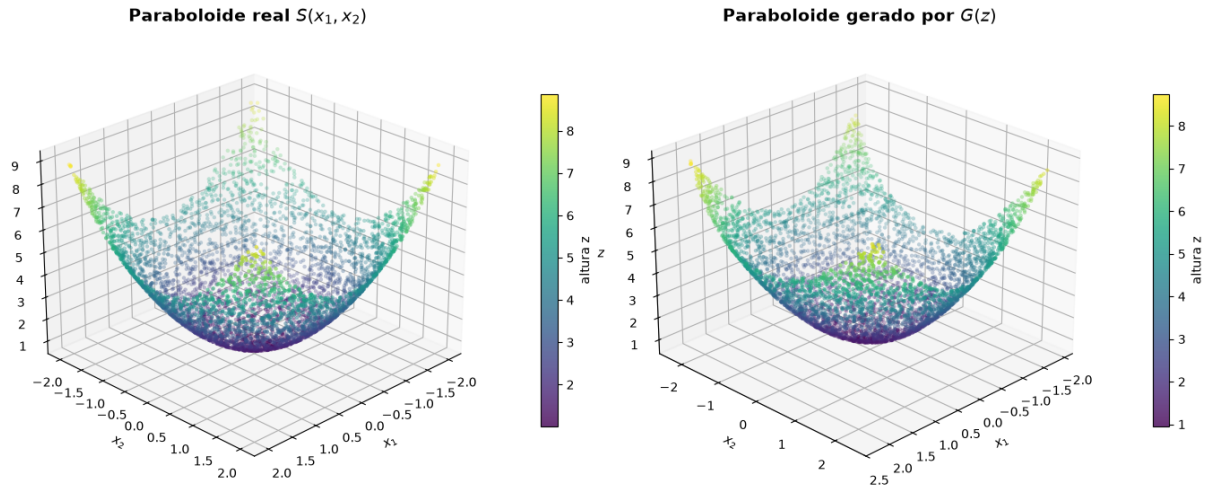


Figura 4: Comparação entre paraboloide do dataset e o gerado pela rede.

Com o intuito de analisar os resultados obtidos, novos vetores do espaço latente foram amostrados e fornecidos à rede Geradora para a criação de novas amostras. Como a GAN aprende a distribuição dos dados e não uma correspondência direta entre entrada e saída, a avaliação não foi realizada comparando pontos individuais gerados com pontos reais, mas sim verificando o erro geométrico das amostras geradas em relação à superfície esperada do parabolóide.

Para isso, foi calculado o erro da superfície gerada utilizando as métricas *Mean Absolute Error* (MAE), definida por:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|e_i|\,, \tag{13}$$

e *Root Mean Squared Error* (RMSE), definida como:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}e_i^2}, \tag{14}$$

onde e_i representa o erro geométrico entre o ponto gerado e a superfície ideal do parabolóide, dado por:

$$e_i = z_i - (x_i^2 + y_i^2 + 1). \tag{15}$$

Além disso, foram calculadas as versões relativas dessas métricas, normalizando os erros pelo valor médio da superfície gerada.

A rede Geradora apresentou um erro geométrico absoluto de aproximadamente 0.35 para a métrica MAE e 0.44 para a métrica RMSE. Quando normalizados, esses valores correspondem a erros relativos de aproximadamente 14.05% e 16.87%, respectivamente. Esses resultados indicam que a GAN foi capaz de aprender a estrutura geral do parabolóide, gerando amostras próximas à superfície real, embora ainda apresente um desvio geométrico devido à dificuldade inerente do treinamento adversarial.

Por fim, considere a parametrização analítica do parabolóide definida pela função $S : \mathbb{R}^2 \to \mathbb{R}^3$,

$$S(x, y) = \left(x, y, x^2 + y^2 + 1\right). \tag{16}$$

Como essa função é diferenciável, seu jacobiano pode ser obtido analiticamente por

$$\nabla S(x, y) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2x & 2y \end{bmatrix}. \tag{17}$$

Da mesma forma, a rede Geradora define uma transformação $G : \mathcal{Z} \to \mathbb{R}^3$, que recebe vetores do espaço latente $\mathcal{Z}$ e produz pontos no espaço tridimensional. Seu jacobiano pode ser obtido por diferenciação automática, sendo dado por

$$\nabla G(z) = \frac{\partial G(z)}{\partial z}. \tag{18}$$

A comparação entre ∇S e ∇G permite investigar se o gerador aprendeu uma parametrização semelhante à função analítica do parabolóide. Caso isso ocorresse, seria esperado que as duas primeiras coordenadas produzidas pelo gerador reproduzissem aproximadamente as coordenadas do vetor latente, enquanto a terceira coordenada representasse a função $x^2 + y^2 + 1$. Entretanto, o treinamento de uma GAN não impõe essa restrição. O objetivo do gerador é apenas transformar vetores do espaço latente em amostras cuja distribuição seja semelhante à distribuição dos dados reais, não existindo qualquer obrigação de preservar as coordenadas do espaço latente durante essa transformação.

A Tabela 1 apresenta a comparação entre os jacobianos da função analítica e da rede Geradora. Para quantificar a diferença entre os jacobianos, foi utilizada a norma de Frobenius da matriz diferença entre ∇G e ∇S. Para cada ponto avaliado, essa métrica é definida por

$$\|\nabla G - \nabla S\|_F = \sqrt{\sum_{i=1}^{3} \sum_{j=1}^{2} \left(\frac{\partial G_i}{\partial z_j} - \frac{\partial S_i}{\partial x_j} \right)^2}, \tag{19}$$

em que a norma de Frobenius corresponde à raiz quadrada da soma dos quadrados de todos os elementos da matriz diferença entre os jacobianos. Quanto menor esse valor, maior é a semelhança entre as transformações locais realizadas pela função analítica e pela rede Geradora. Como essa métrica é calculada para cada amostra avaliada, foi considerada a média dessas normas ao longo de todos os pontos analisados, obtendo-se uma medida global da discrepância entre os dois jacobianos.

A Tabela 1 apresenta a comparação entre os jacobianos médios da função analítica S e da rede Geradora G. A norma média de Frobenius da diferença entre esses jacobianos foi igual a 2,12, indicando que a transformação aprendida pela rede apresenta diferenças em relação à parametrização analítica.

Tabela 1: Comparação entre os jacobianos da função analítica S e da rede Geradora G.

	$\partial x/\partial a$	$\partial x/\partial b$	$\partial y/\partial a$	$\partial y/\partial b$	$\partial z/\partial a$	$\partial z/\partial b$
$G(a,b)$	0.0010	-0.9281	-1.1413	-0.0334	-0.0333	-0.2426
$S(a,b)$	1.0000	0.0000	0.0000	1.0000	-0.0546	-0.0631
$G-S$	-0.9990	-0.9281	-1.1413	-1.0334	0.0213	-0.1795

Métrica	Valor
Norma média de Frobenius	2.1233
Desvio padrão	0.1006

Observa-se que as maiores discrepâncias ocorrem nas derivadas das duas primeiras coordenadas. Enquanto a parametrização analítica possui $\partial x/\partial a = 1$ e $\partial y/\partial b = 1$, a rede Geradora apresentou valores médios de aproximadamente $0,00$ e $-0,03$, respectivamente. Da mesma forma, as derivadas cruzadas, que são identicamente nulas na função analítica, assumiram valores médios elevados na rede Geradora, como $\partial x/\partial b = -0,93$ e $\partial y/\partial a = -1,14$.

Em contrapartida, as derivadas da terceira coordenada apresentaram maior proximidade com a parametrização analítica. Os valores médios obtidos para a rede Geradora foram $\partial z/\partial a = -0,03$ e $\partial z/\partial b = -0,24$, enquanto a função analítica apresentou, respectivamente, $-0,05$ e $-0,06$. Embora ainda exista discrepância, principalmente em $\partial z/\partial b$, ela é inferior à observada nas derivadas das duas primeiras coordenadas.

Esse comportamento pode ser visualizado na Figura 5, que ilustra a correspondência entre os vetores do espaço latente e as duas primeiras coordenadas produzidas pela rede Geradora. Observa-se que as coordenadas geradas não preservam a posição original dos vetores no espaço latente, evidenciando que a rede não aprendeu a identidade entre o espaço latente e as duas primeiras coordenadas da superfície. Em vez disso, o gerador aprendeu uma transformação do espaço latente, utilizando os vetores latentes como fonte de variabilidade para produzir amostras pertencentes à distribuição dos dados. Esse comportamento está de acordo com o objetivo do treinamento adversarial de uma GAN, cujo propósito é aproximar a distribuição dos dados reais, e não reproduzir uma parametrização específica da superfície.

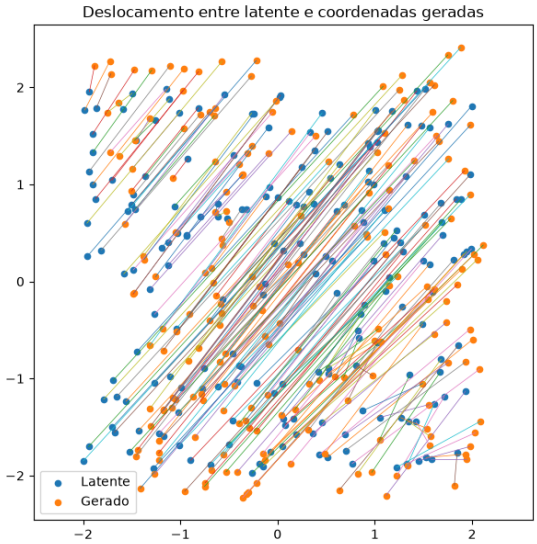


Figura 5: Correspondência entre os vetores do espaço latente e as duas primeiras coordenadas produzidas pela rede Geradora. As setas indicam o deslocamento de cada vetor latente para as coordenadas (x, y) geradas, evidenciando que a rede aprende uma transformação do espaço latente em vez de preservar sua parametrização.

Essa observação também ajuda a explicar as diferenças encontradas entre os jacobianos da função analítica e da rede Geradora. Como a parametrização aprendida pela GAN não precisa coincidir com a parametrização utilizada para gerar os dados, diferentes transformações do espaço latente podem produzir a mesma superfície no espaço de saída. Consequentemente, mesmo que a geometria global do parabolóide seja preservada, os gradientes em relação às variáveis latentes podem diferir daqueles obtidos pela parametrização analítica.

Além disso, reproduzir a geometria diferencial da superfície constitui uma tarefa mais exigente do que apenas aproximar sua distribuição de pontos. A função objetivo de uma GAN clássica não impõe restrições explícitas para que o gerador preserve propriedades diferenciais do *manifold*, tais como gradientes, espaços tangentes ou curvaturas. Dessa forma, mesmo quando as amostras geradas pertencem aproximadamente ao parabolóide, pequenas deformações locais na parametrização podem resultar em diferenças significativas entre os jacobianos da função analítica e da rede Geradora [5].

2 GANs com Operações Convolucionais

Questão 2: Escolha uma base de dados de imagens e treine um modelo GAN sobre ela, com um gerador utilizando operações convolucionais.

(a) Considere dois critérios de parada diferentes e compare-os;

(b) Escolha duas métricas de qualidade de imagem e compare-as para analisar o resultado da GAN.

Resolução:

Para a realização deste experimento, foi utilizado o conjunto de dados *MNIST* (*Modified National Institute of Standards and Technology*) [4], composto por imagens em escala de cinza de dígitos manuscritos. Cada imagem possui dimensão de 28×28 pixels e pertence a uma das dez classes correspondentes aos dígitos de 0 a 9. O conjunto de dados contém 70.000 imagens, sendo 60.000 destinadas ao treinamento e 10.000 ao teste. A Figura 6 apresenta alguns exemplos de imagens presentes na base de dados. Além disso, através da Figura 7, podemos observar que o conjunto de dados é aproximadamente balanceado em relação ao número de amostras de cada classe disponíveis para treinamento.

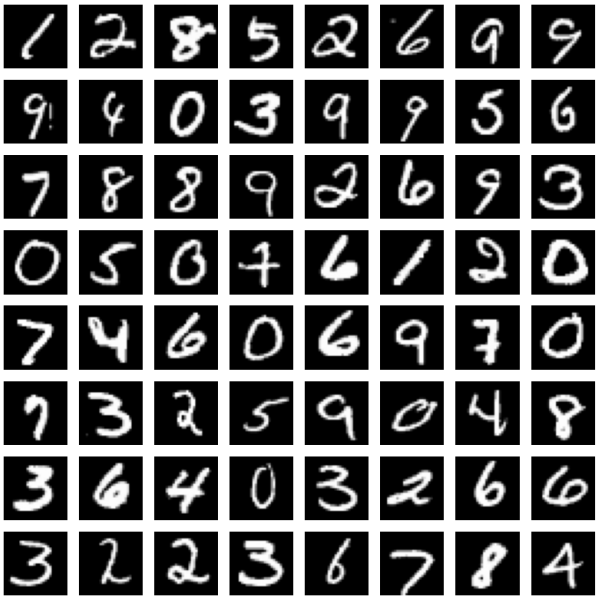


Figura 6: Exemplos de imagens do conjunto de treinamento do MNIST.

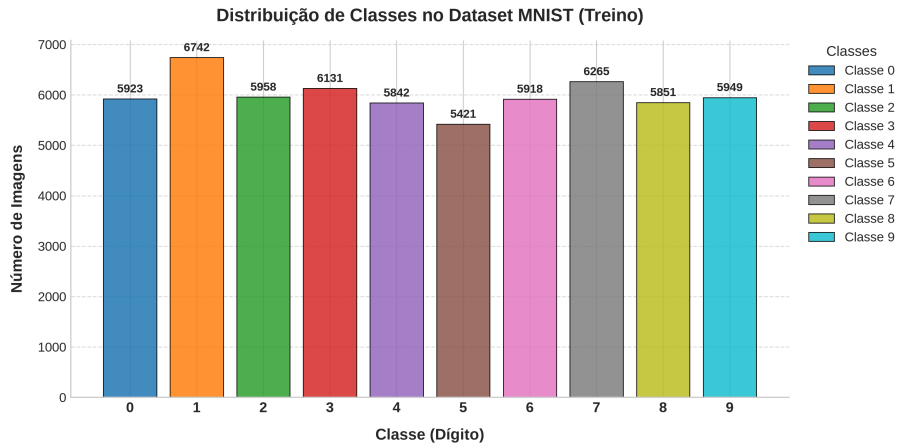


Figura 7: Distribuição dos dados de treinamento por classes.

Como a camada de saída da rede geradora utilizará a função de ativação *Tanh*, cujos valores pertencem ao intervalo $[-1, 1]$, as imagens do MNIST, originalmente representadas no intervalo $[0, 1]$, foram normalizadas para o intervalo $[-1, 1]$ por meio da seguinte transformação:

$$x_{\text{norm}} = 2x - 1. \tag{20}$$

Considerando que o problema consiste na geração de imagens, foi adotada uma arquitetura convolucional para as redes da GAN, inspirada na *Deep Convolutional Generative Adversarial Network* (DCGAN), proposta por [7]. Embora não reproduza exatamente a arquitetura original, o modelo empregado neste trabalho segue os principais princípios da DCGAN, como o uso de camadas convolucionais transpostas no gerador, camadas convolucionais no discriminador e funções de ativação apropriadas para cada rede. A arquitetura utilizada é apresentada na Figura 8.

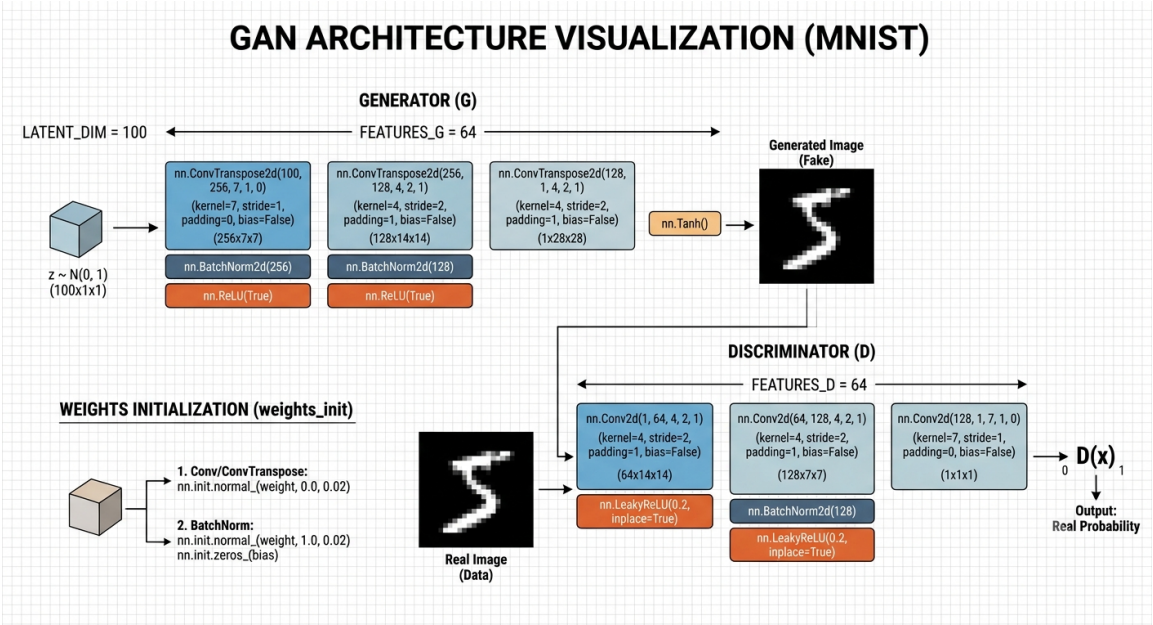


Figura 8: Arquitetura da GAN convolucional utilizada neste trabalho.

A rede Geradora recebe como entrada um vetor de ruído aleatório de dimensão 100, representado no

formato $(100, 1, 1)$. A partir desse vetor, sucessivas camadas de convolução transposta (*ConvTranspose2d*) promovem a expansão gradual da resolução espacial até produzir uma imagem sintética de dimensão 28×28. Entre essas camadas são utilizadas operações de *Batch Normalization* e a função de ativação ReLU, enquanto a camada de saída emprega a função *Tanh*, restringindo os valores gerados ao intervalo $[-1, 1]$, compatível com a normalização aplicada às imagens de treinamento.

Por sua vez, a rede Discriminadora recebe como entrada imagens reais ou sintéticas de dimensão 28×28 e aplica sucessivas operações de convolução para reduzir sua resolução espacial e extrair características discriminativas. Nas camadas intermediárias são utilizadas a função de ativação *LeakyReLU*, com coeficiente negativo igual a 0,2, e camadas de *Batch Normalization*. A camada de saída produz um único valor escalar que representa a probabilidade estimada de a imagem pertencer ao conjunto de dados reais.

A Tabela 2 apresenta a arquitetura completa das redes Geradora e Discriminadora implementadas neste trabalho. Os hiperparâmetros adotados, incluindo o número de filtros, os tamanhos dos *kernels* e os demais parâmetros das camadas convolucionais, foram definidos empiricamente a partir de experimentos com diferentes configurações.

Tabela 2: Arquitetura da GAN utilizada nos experimentos.

Rede	**Camada**	**Filtros**	**Kernel**	**Stride**	**Padding**
Geradora	ConvTranspose2d	256	7×7	1	0
	ConvTranspose2d	128	4×4	2	1
	ConvTranspose2d	1	4×4	2	1
Discriminadora	Conv2d	64	4×4	2	1
	Conv2d	128	4×4	2	1
	Conv2d	1	7×7	1	0

Na rede Geradora, as duas primeiras camadas de convolução transposta são seguidas por operações de *Batch Normalization* e pela função de ativação ReLU. A camada de saída utiliza a função *Tanh*, restringindo os valores gerados ao intervalo $[-1, 1]$.

Na rede Discriminadora, a primeira camada convolucional é seguida pela função de ativação *LeakyReLU*, enquanto a segunda camada é seguida por operações de *Batch Normalization* e pela mesma função de ativação. A camada de saída não emprega função de ativação, produzindo diretamente um *logit*, que é utilizado pela função de perda durante o treinamento.

O treinamento foi realizado de maneira análoga ao descrito na Questão 1, utilizando o otimizador Adam com os mesmos hiperparâmetros e as mesmas funções de custo para o gerador e o discriminador.

Considerando um limite máximo de 100 épocas, foram avaliadas duas estratégias distintas de parada antecipada, descritas a seguir.

- **(M1) Early Stopping baseado no FID:** nesta estratégia, a qualidade das imagens geradas foi monitorada por meio da métrica *Fréchet Inception Distance* (FID), definida por

$$\mathrm{FID}(\mathbf{x}, \mathbf{g}) = \left\| \boldsymbol{\mu}_x - \boldsymbol{\mu}_g \right\|_2^2 + \mathrm{Tr}\left(\Sigma_x + \Sigma_g - 2(\Sigma_x \Sigma_g)^{1/2} \right), \tag{21}$$

  onde (μ_x, Σ_x) representam, respectivamente, o vetor média e a matriz de covariância das representações obtidas a partir das imagens reais, enquanto (μ_g, Σ_g) correspondem às mesmas estatísticas calculadas para as imagens geradas. Neste trabalho, as representações foram obtidas aplicando uma operação de *Average Pooling* às imagens, seguida da vetorização dos mapas resultantes. Quanto

menor o valor do FID, maior a similaridade entre as distribuições das representações das imagens reais e sintéticas.

O treinamento era interrompido caso o melhor valor de FID não apresentasse melhoria durante uma janela consecutiva de $n = 10$ épocas.

- **(M2) Estabilidade da acurácia do discriminador:** nesta estratégia, foi monitorada a acurácia do discriminador sobre o conjunto de validação ao final de cada época. A hipótese adotada é que, quando essa acurácia passa a oscilar dentro de uma faixa reduzida por um período suficientemente longo, o treinamento pode ter atingido um estado próximo do equilíbrio entre gerador e discriminador.

  Para verificar essa condição, foi considerada uma janela móvel contendo as últimas 10 épocas. O treinamento era encerrado quando a diferença entre a maior e a menor acurácia observadas nessa janela fosse inferior a 10%, caracterizando um comportamento suficientemente estável do discriminador.

Após definir os critérios de parada, a GAN foi treinada utilizando cada uma das estratégias propostas. A Figura 9 apresenta a evolução das funções de custo normalizadas do gerador e do discriminador durante o treinamento. Observa-se que a estratégia M1 interrompeu o treinamento na época 26, momento em que o critério de estagnação do FID foi satisfeito. Em contrapartida, utilizando a estratégia M2, o treinamento prosseguiu até a época 59, quando foi detectada a estabilidade da acurácia do discriminador.

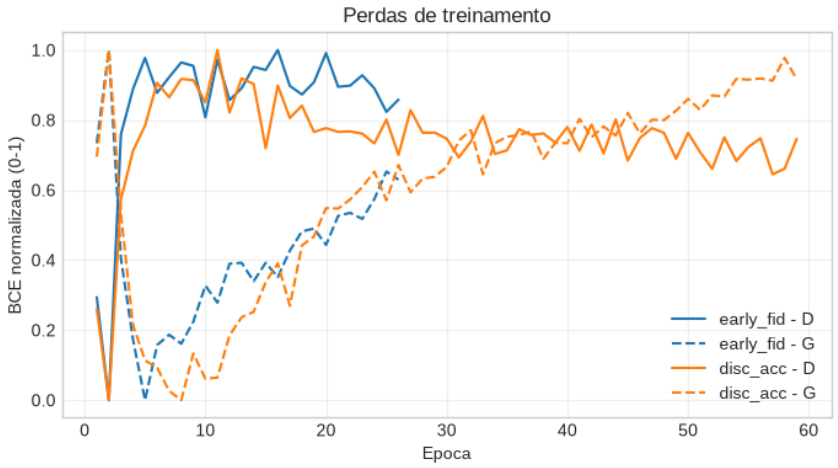


Figura 9: Evolução das funções de custo normalizadas do gerador e do discriminador durante o treinamento da GAN para as duas estratégias de parada antecipada avaliadas: Early FID (M1) e estabilidade da acurácia do discriminador (M2). Observa-se que o treinamento utilizando M1 foi interrompido na época 26, enquanto o treinamento utilizando M2 prosseguiu até a época 59.

Para analisar o comportamento de ambos os critérios, a Figura 10 apresenta a evolução do FID ao longo das épocas, enquanto a Figura 11 mostra a evolução da acurácia do discriminador. Observando a Figura 10, nota-se que a estratégia M1 interrompeu o treinamento quando o FID permaneceu sem melhora durante a janela de monitoramento estabelecida. Entretanto, ao analisar a curva correspondente ao treinamento utilizando M2, percebe-se que, após algumas épocas adicionais, o FID voltou a diminuir e atingiu valores inferiores ao obtido pela estratégia M1. Esse comportamento indica que, para os hiperparâmetros utilizados, o critério de parada baseado exclusivamente na estagnação do FID mostrou-se excessivamente conservador, interrompendo o treinamento antes que o gerador atingisse sua melhor qualidade.

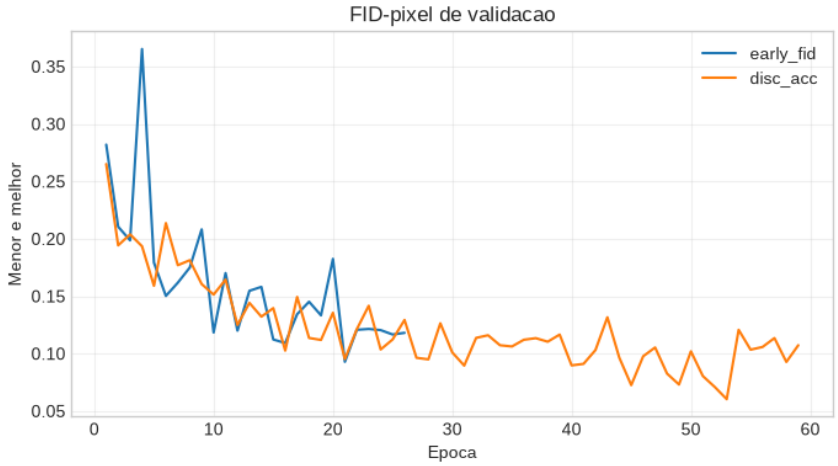


Figura 10: Evolução da métrica Fréchet Inception Distance (FID) ao longo do treinamento para as estratégias Early FID (M1) e estabilidade da acurácia do discriminador (M2). Valores menores indicam maior similaridade entre as distribuições das imagens reais e geradas.

Por outro lado, a Figura 11 permite interpretar o comportamento da estratégia M2. Entre aproximadamente as épocas 30 e 50, a acurácia do discriminador apresentou oscilações relativamente elevadas, indicando que o processo adversarial ainda estava em evolução. Somente após esse período a acurácia passou a permanecer dentro da faixa de estabilidade estabelecida, levando ao encerramento do treinamento. Embora os valores absolutos da acurácia nesse momento sejam relativamente elevados, sugerindo que o discriminador ainda conseguia distinguir parte das imagens sintéticas, a evolução do FID demonstra que o gerador continuou melhorando sua capacidade de produzir amostras realistas durante esse intervalo.

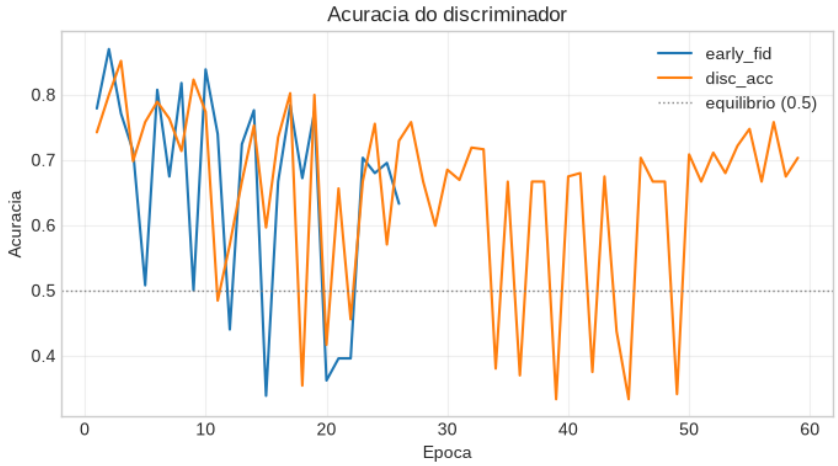
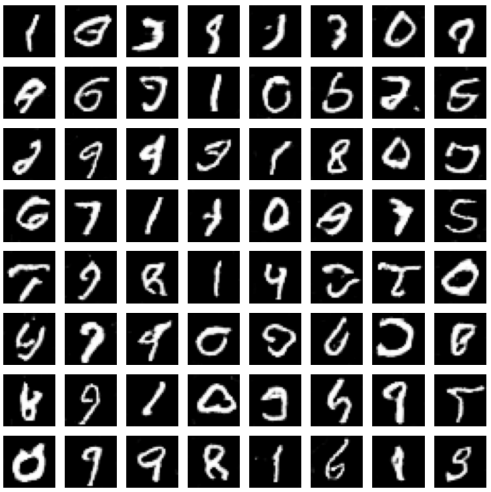


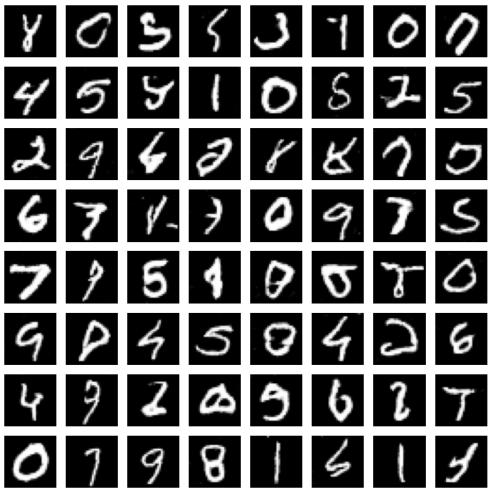
Figura 11: Evolução da acurácia do discriminador no conjunto de validação durante o treinamento da GAN para as estratégias Early FID (M1) e estabilidade da acurácia do discriminador (M2). A região final da curva de M2 apresenta menor variação entre épocas, satisfazendo o critério de estabilidade utilizado para a parada antecipada.

A Figura 12 apresenta exemplos de imagens sintetizadas pelos modelos obtidos por meio das diferentes estratégias de parada. Visualmente, observa-se uma ligeira superioridade das imagens produzidas pela estratégia M2, que apresentam menor quantidade de artefatos no fundo e dígitos com contornos mais consistentes. Entretanto, as diferenças qualitativas entre os modelos são sutis e podem não ser facilmente perceptíveis apenas pela inspeção visual, o que reforça a importância da utilização de métricas

quantitativas, como FID e MMD, para uma avaliação mais objetiva da qualidade das imagens geradas.



(a) Imagens geradas pelo modelo treinado utilizando o critério de parada antecipada baseado no FID (M1).



(b) Imagens geradas pelo modelo treinado utilizando o critério de estabilidade da acurácia do discriminador (M2).

Figura 12: Exemplos de dígitos sintéticos produzidos pelos modelos obtidos utilizando os dois critérios de parada antecipada. Visualmente, observa-se que o modelo treinado com o critério M2 produz amostras mais consistentes, com menor quantidade de artefatos e melhor definição dos dígitos.

Como referência adicional, o modelo também foi treinado durante as 100 épocas previstas, sem a utilização de parada antecipada. Para a comparação entre os modelos, além da métrica Fréchet Inception Distance (FID), foi utilizada a métrica *Maximum Mean Discrepancy* (MMD), que mede a distância entre duas distribuições de probabilidade por meio de uma função de kernel. Considerando dois conjuntos de amostras $X = \{x_i\}_{i=1}^{n}$ e $Y = \{y_j\}_{j=1}^{m}$, a MMD é definida por

$$\mathrm{MMD}^2(X, Y) = \frac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n} k(x_i, x_j) + \frac{1}{m^2}\sum_{i=1}^{m}\sum_{j=1}^{m} k(y_i, y_j) - \frac{2}{nm}\sum_{i=1}^{n}\sum_{j=1}^{m} k(x_i, y_j), \tag{22}$$

em que $k(\cdot, \cdot)$ representa uma função de kernel, sendo utilizado neste trabalho o kernel Gaussiano (RBF). Assim como ocorre com o FID, valores menores de MMD indicam maior similaridade entre as distribuições das imagens reais e geradas.

A Tabela 3 resume os resultados quantitativos obtidos utilizando as métricas FID e MMD para os três modelos treinados.

Tabela 3: Comparação entre os diferentes critérios de parada.

Estratégia	**Épocas**	**FID ↓**	**MMD ↓**
Treinamento completo	100	0.1075	0.00213
Early FID (M1)	26	0.1164	0.00379
Estabilidade do discriminador (M2)	59	**0.0727**	**0.00142**

Os resultados quantitativos corroboram a análise visual. Embora o treinamento completo tenha utilizado todas as 100 épocas disponíveis, ele não apresentou o melhor desempenho. A estratégia M2 obteve simultaneamente os menores valores de FID e MMD, indicando maior proximidade entre as distribuições das imagens reais e geradas. Por outro lado, a estratégia M1 apresentou os maiores valores em ambas as

métricas, sugerindo que o critério de parada baseado exclusivamente na estagnação do FID interrompeu o treinamento prematuramente.

De maneira geral, os resultados obtidos pelas duas métricas são consistentes entre si e indicam que a utilização da estabilidade da acurácia do discriminador como critério de parada foi a estratégia mais eficaz para este problema. Essa abordagem permitiu que a rede Geradora continuasse evoluindo mesmo após períodos temporários de estagnação do FID, produzindo ao final do treinamento imagens com maior similaridade estatística em relação ao conjunto de dados real.

3 CNN para Estimativa de Idade

Questão 3: Utilize a base de dados UTKFace (`https://github.com/aicip/UTKFace`) ou outro repositório similar, e considere o problema de regressão de estimar a idade dada uma imagem facial usando um modelo CNN. Aplique teoria de invariância (ou equivariância) para melhorar os resultados.

Resolução:

Na realização desta questão foi utilizado o conjunto de dados *UTKFace*, composto por 23.708 imagens RGB de faces humanas abrangendo diferentes idades, gêneros e etnias. Cada imagem possui como rótulo a idade da pessoa retratada, permitindo que o problema seja formulado como uma tarefa de regressão.

A Figura 13 apresenta a distribuição das amostras por faixas etárias. Observa-se um desbalanceamento significativo no conjunto de dados, caracterizado por uma elevada concentração de exemplos entre 20 e 29 anos, enquanto as faixas etárias mais elevadas, especialmente acima dos 80 anos, possuem um número substancialmente menor de amostras.

Esse desbalanceamento pode influenciar diretamente o processo de aprendizado da rede, uma vez que modelos supervisionados tendem a ajustar seus parâmetros de forma mais eficiente em regiões do espaço de dados com maior representatividade. Como consequência, espera-se um desempenho superior para as faixas etárias mais frequentes e um aumento do erro de predição para idades pouco representadas, devido à menor quantidade de exemplos disponíveis para aprendizagem.

Nesta etapa, optou-se por não empregar técnicas de *data augmentation* do tipo *offline*, nas quais novas imagens transformadas são previamente geradas e incorporadas ao conjunto de treinamento. Essas técnicas poderiam ser utilizadas para aumentar a representatividade das faixas etárias menos frequentes, tornando a distribuição das amostras mais equilibrada. Dessa forma, optou-se por manter o conjunto de dados em sua forma original, a fim de avaliar o impacto do desbalanceamento no processo de aprendizado. Nos resultados, será realizada uma análise do desempenho do modelo em cada faixa etária, permitindo verificar se a distribuição desigual de amostras influenciou sua capacidade de predição.

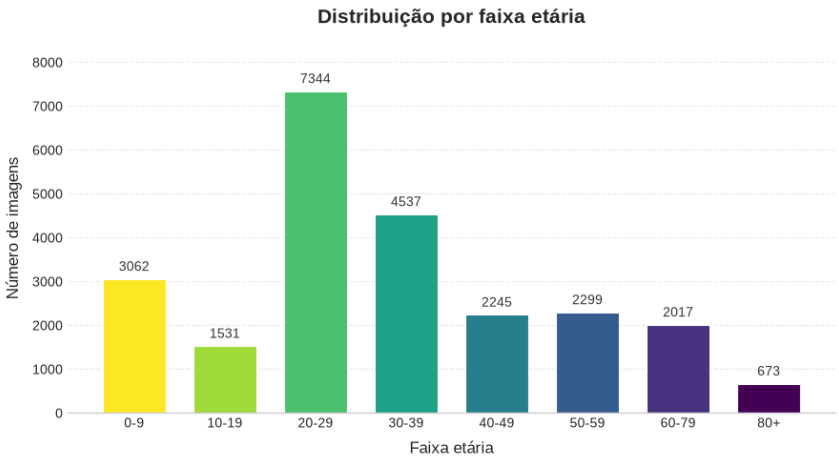


Figura 13: Distribuição do número de imagens do conjunto *UTKFace* agrupadas por faixas etárias. Observa-se uma concentração de amostras em adultos jovens, principalmente entre 20 e 29 anos, enquanto as faixas etárias mais avançadas apresentam quantidade significativamente menor de exemplos.

O conjunto de dados foi dividido em três subconjuntos independentes, sendo 16.595 imagens destinadas ao treinamento, 1.556 imagens para validação e 3.557 imagens para teste. O conjunto de treinamento foi utilizado para o ajuste dos parâmetros da rede neural, enquanto o conjunto de validação foi empregado para monitorar o desempenho do modelo ao longo do treinamento e selecionar o modelo com melhor capacidade de generalização. Por fim, o conjunto de teste foi utilizado exclusivamente para a avaliação final, fornecendo uma estimativa imparcial do desempenho do modelo em dados não observados durante o treinamento.

Antes de serem fornecidas à rede, todas as imagens foram normalizadas canal a canal de acordo com

$$x' = \frac{x - \mu}{\sigma}, \tag{23}$$

em que $\mu = 0{,}5$ e $\sigma = 0{,}5$ para os três canais RGB. Como os pixels já estavam representados no intervalo $[0, 1]$, essa transformação mapeia seus valores para o intervalo $[-1, 1]$, tornando a distribuição dos dados mais adequada ao treinamento da rede neural e contribuindo para a estabilidade do processo de otimização.

Como modelo de regressão foi utilizada uma rede neural convolucional (CNN), cuja arquitetura é composta por dois módulos principais. O primeiro consiste em uma sequência de blocos convolucionais responsáveis pela extração automática de características das imagens de entrada. Cada bloco é formado por uma camada convolucional, seguida pela função de ativação ReLU e por uma operação de redução da resolução espacial, permitindo que a rede aprenda representações hierárquicas das características faciais.

Após a etapa de extração de características, é aplicado um operador de *Adaptive Average Pooling*, responsável por reduzir os mapas de características a uma representação espacial de tamanho fixo. Em seguida, essa representação é convertida em um vetor e processada por camadas totalmente conectadas (*fully connected*), responsáveis por estimar a idade da pessoa presente na imagem. Para reduzir o risco de sobreajuste durante o treinamento, foi empregada uma camada de *dropout* com taxa de 30%.

A Figura 14 apresenta a arquitetura da rede neural convolucional utilizada neste trabalho, destacando os blocos convolucionais responsáveis pela extração automática de características das imagens e as camadas totalmente conectadas encarregadas da estimação da idade.

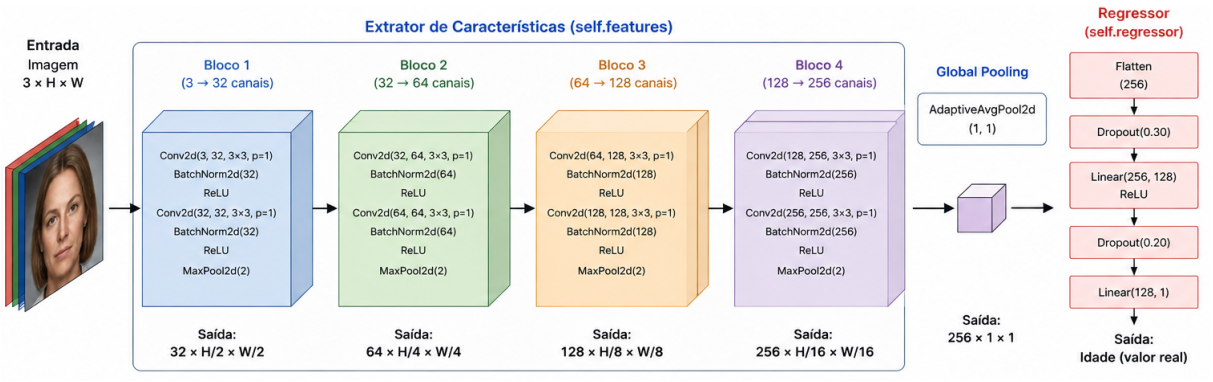


Figura 14: Arquitetura da rede neural convolucional utilizada para a regressão da idade. O modelo é composto por quatro blocos convolucionais para extração de características, seguidos por uma etapa de *Adaptive Average Pooling* e camadas totalmente conectadas que produzem uma única saída correspondente à idade estimada.

O treinamento foi realizado utilizando o algoritmo Adam (*Adaptive Moment Estimation*) com seus hiperparâmetros padrão. A otimização dos parâmetros da rede foi conduzida minimizando a função de custo de regressão adotada, conforme descrito a seguir.

Seja f_θ a rede neural parametrizada pelo conjunto de parâmetros θ. Considerando um mini-lote de treinamento B, a função de custo utilizada foi o Erro Quadrático Médio (*Mean Squared Error – MSE*), definido por

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|B|} \sum_{i=1}^{|B|} \left(y_i - f_\theta(x_i)\right)^2, \tag{24}$$

em que x_i representa a i-ésima imagem do mini-lote, y_i corresponde à idade real associada a essa imagem e $f_\theta(x_i)$ representa a idade estimada pela rede neural. Essa função penaliza quadraticamente o erro de predição, atribuindo maior peso a estimativas mais distantes do valor real. Por essa razão, o MSE é amplamente utilizado em problemas de regressão, incentivando o modelo a reduzir principalmente os erros de maior magnitude.

Para explorar o princípio da invariância durante o treinamento da rede neural, foram avaliadas duas estratégias distintas baseadas na aplicação de transformações às imagens de entrada.

- **Aumentação dos dados:** nesta abordagem, cada imagem de entrada x_i pode ser submetida, de forma aleatória, a um conjunto de transformações antes de ser apresentada à rede. As transformações são realizadas dinamicamente durante o treinamento (*online data augmentation*), de modo que o número de imagens armazenadas no conjunto de dados permanece inalterado. Entretanto, uma mesma imagem pode ser apresentada à rede com diferentes variações ao longo das épocas, aumentando a diversidade efetiva das amostras observadas durante o treinamento. As transformações utilizadas foram:
  - **Recorte aleatório com redimensionamento (*Random Resized Crop*):** realiza um recorte aleatório preservando entre 85% e 100% da área original da imagem, permitindo pequenas variações na razão de aspecto antes do redimensionamento para o tamanho de entrada da rede.
  - **Espelhamento horizontal aleatório (*Random Horizontal Flip*):** inverte horizontalmente a imagem com probabilidade de 50%, aumentando a variabilidade das poses sem alterar a idade da pessoa.

- **Rotação aleatória (*Random Rotation*):** aplica rotações aleatórias de até $\pm 10°$, tornando o modelo mais robusto a pequenas inclinações da cabeça.
- **Variação de brilho, contraste, saturação e tonalidade (*Color Jitter*):** modifica levemente as características fotométricas da imagem, simulando diferentes condições de iluminação e aquisição.

Seja $\mathcal{T}$ uma transformação pertencente ao conjunto descrito anteriormente. Nesse caso, a função de custo passa a ser dada por

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|B|} \sum_{i=1}^{|B|} \left(y_i - f_\theta(\mathcal{T}(x_i))\right)^2 . \tag{25}$$

A aplicação dessas transformações não altera o rótulo associado à imagem, uma vez que pequenas variações de posição, orientação ou iluminação não modificam a idade da pessoa retratada. Dessa forma, a rede é treinada para produzir estimativas consistentes para diferentes versões da mesma imagem, favorecendo o aprendizado de representações invariantes às transformações aplicadas.

Idealmente, espera-se que, ao final do treinamento,

$$f_\theta(\mathcal{T}_1(x_i)) \approx f_\theta(\mathcal{T}_2(x_i)) \approx \cdots \approx f_\theta(\mathcal{T}_n(x_i)) \approx f_\theta(x_i) \approx y_i, \tag{26}$$

em que $\mathcal{T}_1, \ldots, \mathcal{T}_n$ representam diferentes transformações aplicadas à mesma imagem ao longo das épocas.

- **Regularização de Consistência:** diferentemente da abordagem anterior, nesta estratégia a imagem original continua sendo utilizada na função de custo supervisionada. Entretanto, uma versão transformada da mesma imagem é empregada para construir um termo adicional de regularização, cujo objetivo é incentivar a rede a produzir predições consistentes para ambas as versões da imagem.

  A função de custo passa a ser dada por

$$\mathcal{L} = \frac{1}{|B|} \sum_{i=1}^{|B|} \left(y_i - f_\theta(x_i)\right)^2 + \lambda \frac{1}{|B|} \sum_{i=1}^{|B|} \left(f_\theta(\mathcal{T}(x_i)) - f_\theta(x_i)\right)^2 , \tag{27}$$

  em que o primeiro termo corresponde ao erro supervisionado de regressão (MSE), enquanto o segundo corresponde ao termo de regularização de consistência. O hiperparâmetro λ controla a influência desse termo na função de custo e, neste trabalho, foi fixado em $\lambda = 0,5$.

  Dessa forma, além de minimizar o erro em relação ao rótulo verdadeiro, o modelo é incentivado a produzir estimativas semelhantes para uma imagem e para suas versões transformadas. Como consequência, espera-se que a rede aprenda representações mais invariantes às transformações aplicadas, aumentando sua robustez e capacidade de generalização.

A partir das estratégias descritas anteriormente, foram considerados quatro cenários de treinamento, todos executados durante 50 épocas:

- **M1:** treinamento convencional, sem aumentação de dados e sem regularização de consistência;
- **M2:** treinamento utilizando apenas aumentação de dados;
- **M3:** treinamento utilizando apenas regularização de consistência;

- **M4:** treinamento utilizando simultaneamente aumentação de dados e regularização de consistência.

A Figura 15 apresenta a evolução da função de custo durante o treinamento dos quatro modelos. Observa-se que os modelos M1 e M2 atingem menores valores de perda ao longo das épocas em comparação aos modelos M3 e M4. Entretanto, essa comparação deve ser interpretada com cautela, pois as funções objetivo utilizadas não são equivalentes. Enquanto os modelos M1 e M2 minimizam apenas o erro supervisionado de regressão, os modelos M3 e M4 incluem um termo adicional de regularização por consistência. Consequentemente, a perda total desses modelos incorpora tanto o erro de regressão quanto a penalização associada à consistência entre as predições da imagem original e de suas versões transformadas, resultando naturalmente em valores mais elevados.

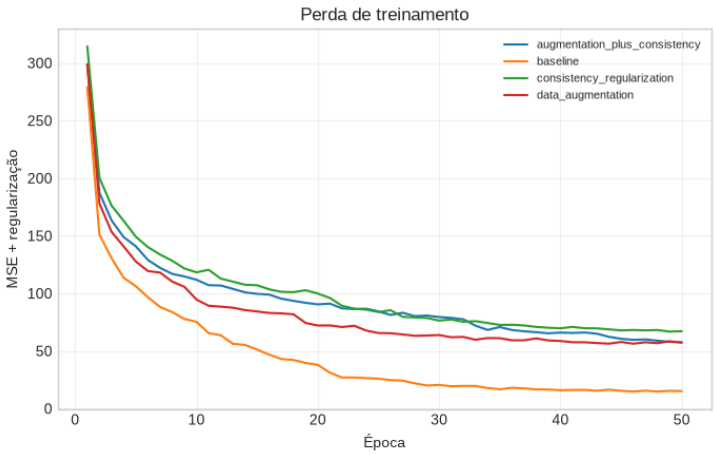


Figura 15: Evolução da função de custo durante o treinamento dos quatro modelos avaliados. Os modelos M3 e M4 apresentam valores mais elevados devido à inclusão do termo de regularização por consistência na função objetivo.

Por outro lado, a Figura 16 apresenta a evolução do Erro Absoluto Médio (MAE, Equação 13) calculado no conjunto de validação. Observa-se que todos os modelos apresentam oscilações relativamente elevadas durante as primeiras épocas de treinamento, comportamento esperado enquanto a rede ainda está ajustando seus parâmetros.

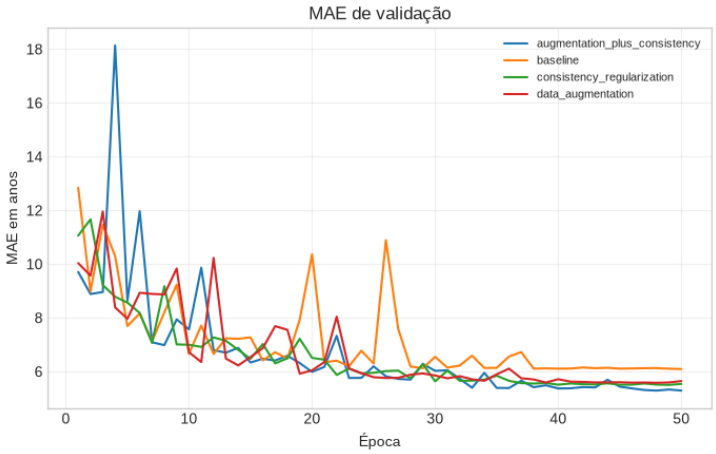


Figura 16: Evolução do Erro Absoluto Médio (MAE) no conjunto de validação durante o treinamento dos modelos M1 (baseline), M2 (aumentação de dados), M3 (regularização de consistência) e M4 (combinação das duas estratégias). Observa-se uma redução progressiva do erro ao longo das épocas, com o modelo M4 apresentando os menores valores ao final do treinamento.

Ainda observando a Figura 16, após aproximadamente a época 30, as curvas tornam-se mais estáveis, permitindo uma comparação mais consistente entre as diferentes estratégias. Ao final do treinamento, o modelo M4 apresenta os menores valores de MAE, enquanto o modelo convencional (M1) apresenta o pior desempenho entre os métodos avaliados.

A Figura 17 apresenta os valores finais de MAE e RMSE (Equação 14) obtidos no conjunto de teste para cada modelo. Os resultados mostram que o modelo M4 obteve simultaneamente os menores valores para ambas as métricas, indicando maior precisão na estimativa da idade. Em contrapartida, o modelo M1 apresentou os maiores erros, evidenciando que a utilização isolada do treinamento supervisionado foi menos eficaz do que as estratégias que incorporam invariância às transformações.

Observa-se ainda que tanto a aumentação de dados quanto a regularização de consistência contribuíram individualmente para reduzir os erros de predição em relação ao modelo de referência (M1). Entretanto, a combinação dessas duas estratégias (M4) proporcionou o melhor desempenho geral, sugerindo que seus efeitos são complementares na tarefa de regressão da idade.

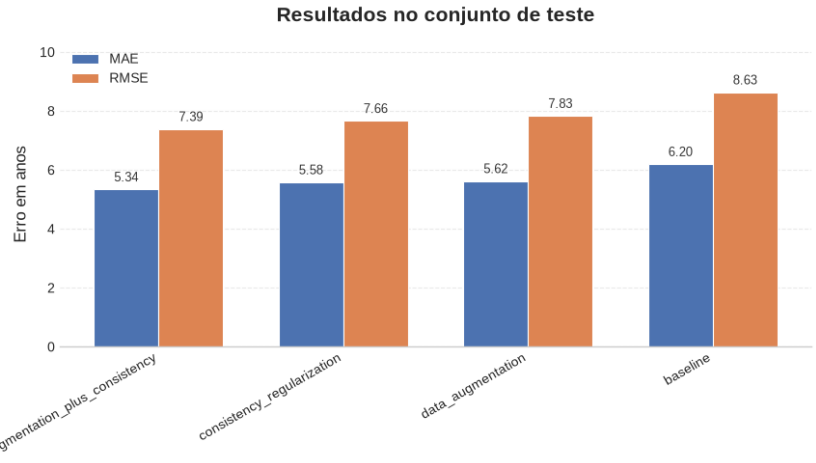


Figura 17: Valores finais de MAE e RMSE obtidos no conjunto de teste para os quatro modelos avaliados. O modelo M4 apresenta os menores valores para ambas as métricas, indicando melhor desempenho na tarefa de regressão da idade.

Além disso, conforme observado na Figura 13, o conjunto de dados apresenta um desbalanceamento significativo entre as diferentes faixas etárias. Nesse contexto, a Figura 18 apresenta os valores de MAE obtidos no conjunto de teste para cada faixa etária. Os resultados mostram uma tendência consistente entre os quatro modelos: as maiores taxas de erro ocorrem justamente nas faixas etárias menos representadas no conjunto de treinamento. Em particular, a classe correspondente às pessoas com 80 anos ou mais, que possui o menor número de amostras, apresentou os maiores valores de MAE. Esses resultados são consistentes com a hipótese de que o desbalanceamento do conjunto de dados dificulta o aprendizado da rede para idades pouco representadas. Porém, podemos observar que os modelos que utilizaram técnicas baseadas na invariância apresentaram melhores resultados para as classes com menos dados disponíveis, evidenciando seu potencial de melhora na generalização do modelo ao se utilizar dessas estratégias.

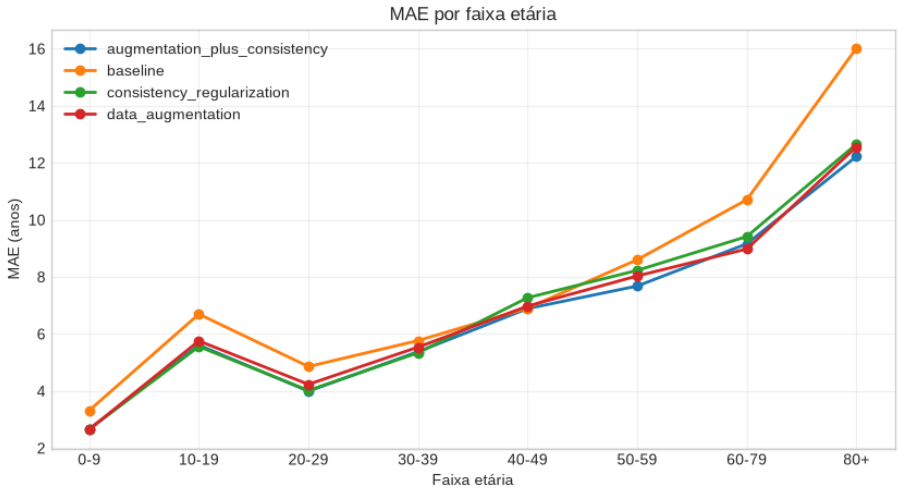


Figura 18: Valores finais de MAE obtidos no conjunto de teste para cada faixa etária.

Por fim, a Figura 19 apresenta exemplos de imagens pertencentes ao conjunto de teste, acompanhadas da idade real e das idades estimadas pelos modelos M1, M2, M3 e M4. De maneira geral, observa-se que os modelos que incorporam o princípio da invariância produzem estimativas mais próximas dos valores reais, com destaque para o modelo M4, que apresenta maior consistência nas predições. Os resultados obtidos demonstram a importância da incorporação do princípio da invariância durante o treinamento da rede neural. Enquanto o modelo M1 foi treinado apenas com o erro supervisionado, os demais modelos exploraram diferentes mecanismos para tornar as predições mais robustas a pequenas variações nas imagens de entrada. Como consequência, todos os modelos baseados em invariância apresentaram desempenho superior ao modelo de referência. Em particular, o modelo M4, que combina aumentação de dados e regularização de consistência, obteve os melhores resultados em todas as métricas analisadas. Comparando as estratégias individualmente, observa-se que a regularização de consistência (M3) apresentou desempenho ligeiramente superior ao obtido apenas com aumentação de dados (M2), embora ambas tenham contribuído para melhorar a capacidade de generalização da rede.

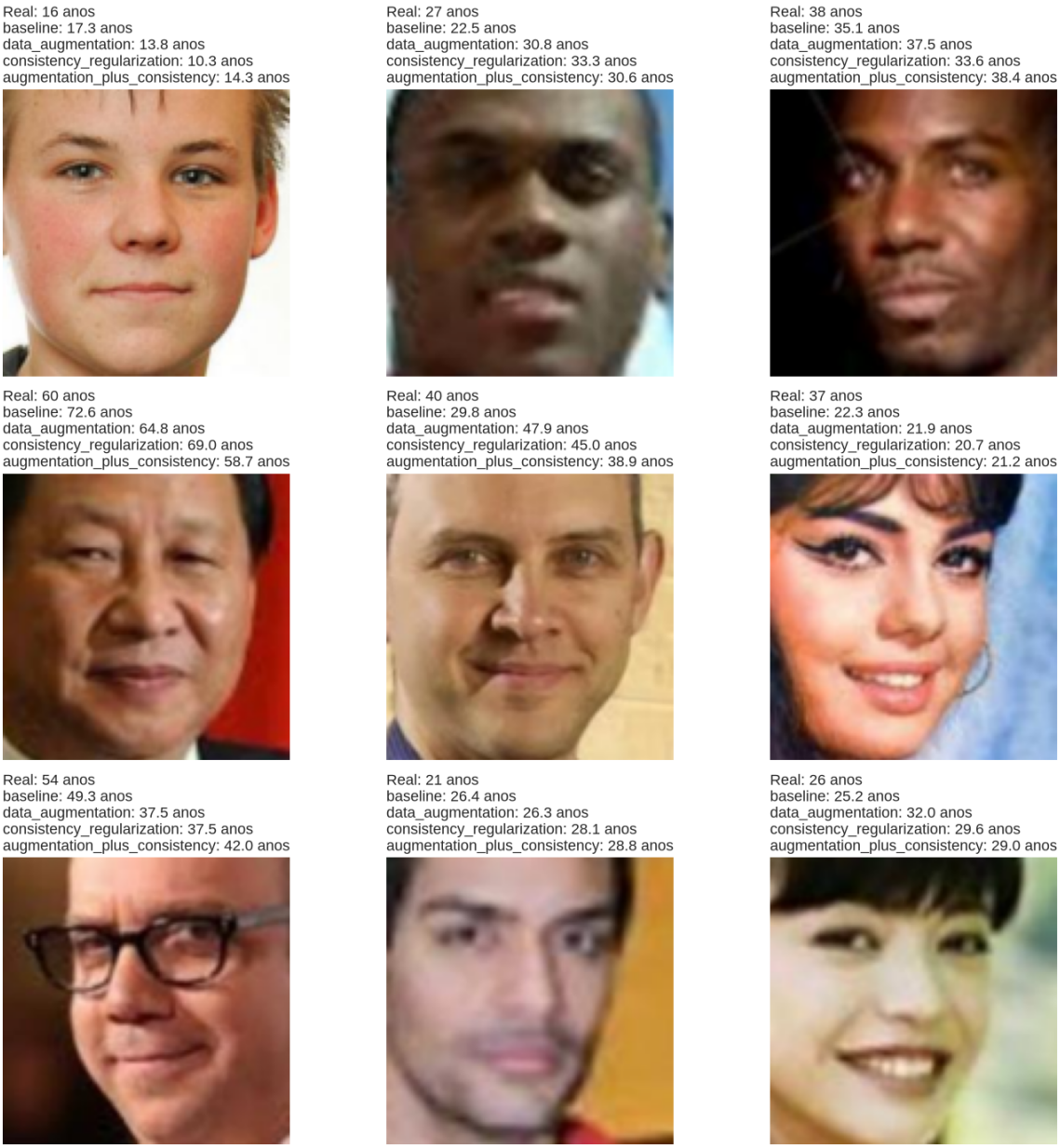


Figura 19: Exemplos de imagens do conjunto de teste. Para cada imagem são apresentados a idade real e as idades estimadas pelos modelos M1 (baseline), M2 (aumentação de dados), M3 (regularização de consistência) e M4 (combinação das duas estratégias).

Referências

[1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

[2] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[3] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9 (3):90–95, 2007.

[4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

[5] Qunwei Li, Bhavya Kailkhura, Rushil Anirudh, Jize Zhang, Yi Zhou, Yingbin Liang, T. Yong Jin Han, and Pramod Varshney. Mr-gan: Manifold regularized generative adversarial networks for scientific data. *SIAM Journal on Mathematics of Data Science*, 3(4):1197–1222, 2021.

[6] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019.

[7] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In *International Conference on Learning Representations*, 2016.